

Compte rendu TP3

Introduction :

Le TP sur lequel nous avons travaillé aujourd'hui reposait sur la communication interne des machines. Comme la machine SPIE étudiée dans le TP précédent, La machine Actémium utilise un bus de terrain nommé « ETERCAT » pour mettre en relation ses équipements.

Présentation :

Cette machine est utilisée dans l'usine afin de remplir des caisses avec des assemblages de pièces. Les pièces arrivent donc imbriquées les unes dans les autres en entrée de machine et ressortent rangées dans des caisses. Au niveau des technologies utilisées, elles sont quasiment identiques que sur la machine SPIE.

Composition machine :

- Convoyeur rouleaux pour caisses
- Convoyeur tapis pour pièces
- Caméra vision
- Trois moteurs pour bras
- IHM
- Afficheurs
- Capteurs intelligents

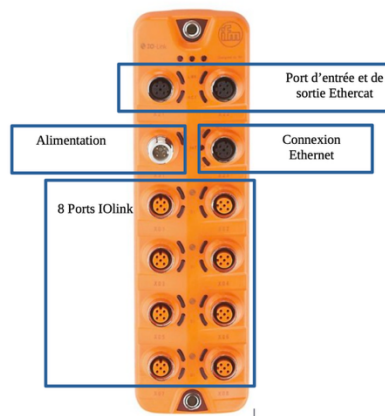


Travail réalisé :

Pour mettre en relation les équipements tels que les variateurs de vitesses, les capteurs intelligents ou encore l'automate, la solution retenue lors de la création de l'usine a été d'utiliser des cartes IOLink. Ces cartes fonctionnent un petit peu à la manière de switches, c'est-à-dire qu'elles servent de multiprise pour les équipements mais on va voir que ça ne se résume pas que à ça.

Maitre IOLink :

Pour éviter d'intervenir sur le câblage de la machine au début nous avons commencé par câbler un maitre IOLink indépendant.



On peut donc voir sur l'image que la carte est composée de plusieurs ports différents. Dans un premier temps, nous avons deux ports Ethercat car on le rappelle le bus de terrain en place sur les machines de l'usine est de l'Ethercat. On a également l'alimentation, un port Ethernet pour la connexion au réseau ainsi que 8 ports IOLink. Ces ports IOLink sont paramétrables de trois manières différentes :

- Entrée TOR
- Sortie TOR
- E/S IOLink

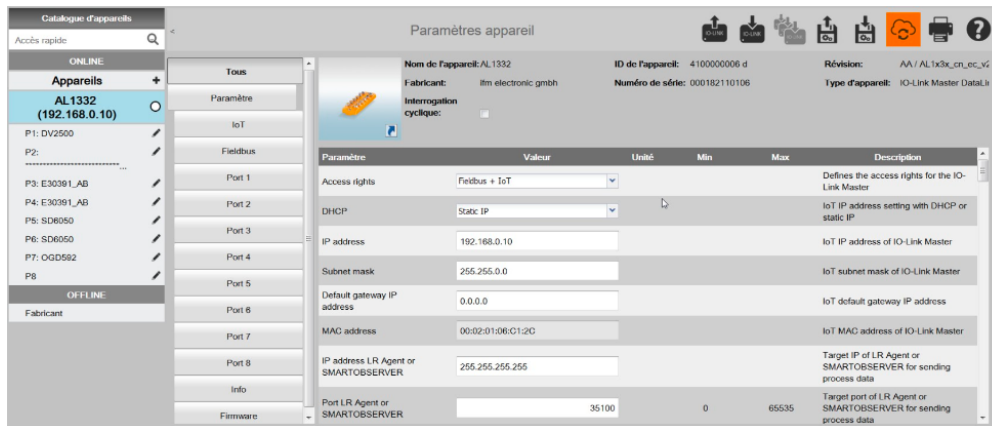
Dans notre cas, il est impératif d'utiliser ces ports en E/S IOLink car nous utilisons des capteurs intelligents qui ne renvoient donc pas simplement 0 ou 1 en cas de détection d'objet.

Pour découvrir la carte nous avons testé deux manières de configurer un capteur intelligent, dans un premier temps avec le logiciel LR DEVICE, puis dans un second avec une application directement depuis notre téléphone. Il était donc possible dans les deux cas de régler un paramètre intéressant qui était la distance à partir de laquelle le capteur devait renvoyer 1.

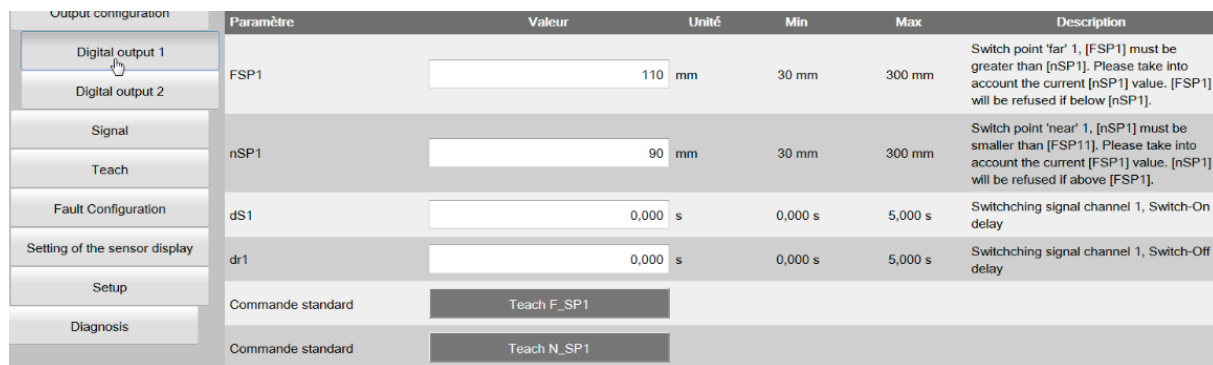
Depuis le logiciel on a commencé par effectuer un scan appareil pour connaître l'IP actuel de la carte sur laquelle nous allons travailler.



On arrive ensuite sur un tableau de bord avec une visu sur tous les ports qu'il est possible de configurer.

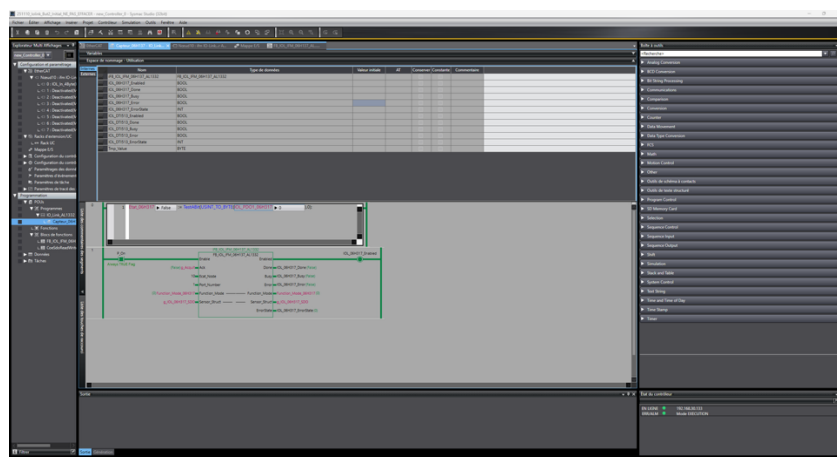


C'est donc pour finir sur cette interface que nous avons pu régler la distance de détection minimum et maximum de notre capteur.

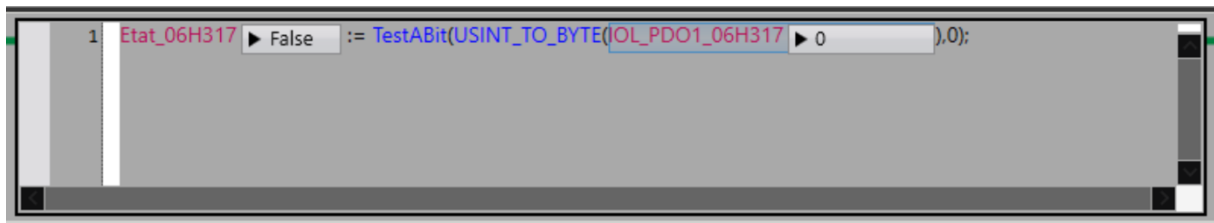


La deuxième étape consistait à configurer le même capteur mais cette fois ci depuis Sysmac Studio. Pour cela on a mappé une variable que l'on récupérer pour afficher ce que renvoie le capteur.

Suite à cela nous avons recopier une ligne de code qui stock l'état du capteur dans une autre variable, on va donc voir à quoi cela sert-il.



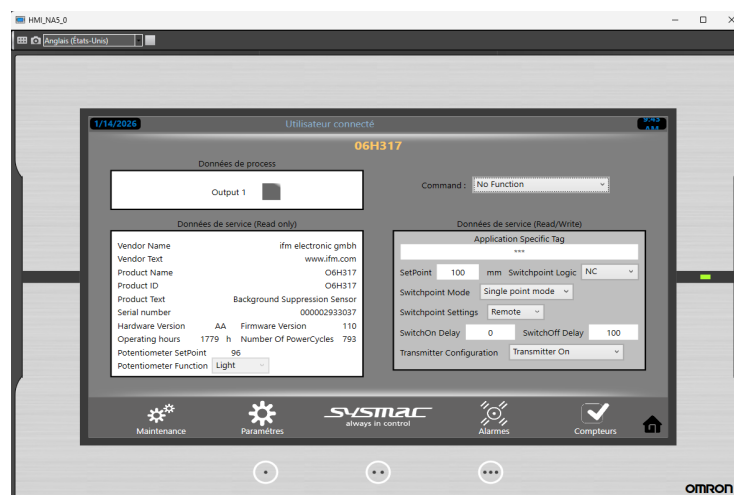
Voici donc la ligne de code :



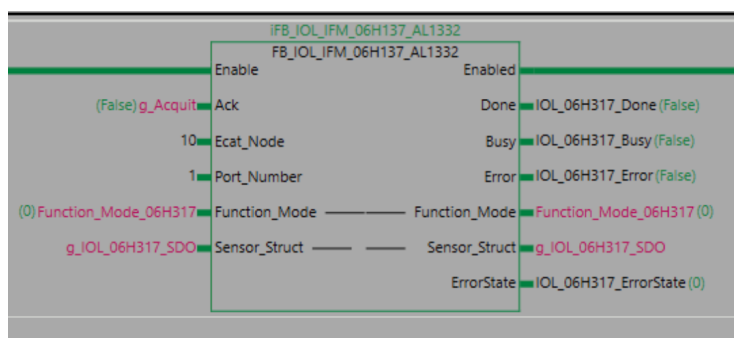
Dans un premier temps nous pouvons voir que des modifications sont en effet apportées à notre variable d'état de capteur de base. La commande « USINT_TO_BYTE » est utilisée pour convertir des USINT en BYTE, ceci est possible car les deux formats occupent la même place dans la mémoire (1 octet), USINT étant le format dans lequel le capteur renvoie directement son état.

La dernière commande que l'on peut lire sur la ligne (« TestABit ») est simplement une commande de lecture de Bit. Le résultat de cette ligne de commande sera donc toujours 0 ou 1.

Pour valider la bonne communication de l'état de la variable, nous avons émulé un IHM et avons observé les changements qui pouvaient apparaître en faisant varier la valeur du capteur.



Pour finir avec cette partie nous avons tester de modifier la distance de commutation du capteur depuis l'IHM pour valider que la communication entre l'IHM et l'automate se fait bien en lecture/écriture et non pas juste en lecture. En revenant d'ailleurs sur le bloc de programme de communication, on voit bien que les variables locales Function_Mode ainsi Sensor_Struct sont directement lié entre l'automate et l'IHM, Sensor_Struct servant justement à ajuster la distance de commutation.



Configuration d'un capteur de distance :

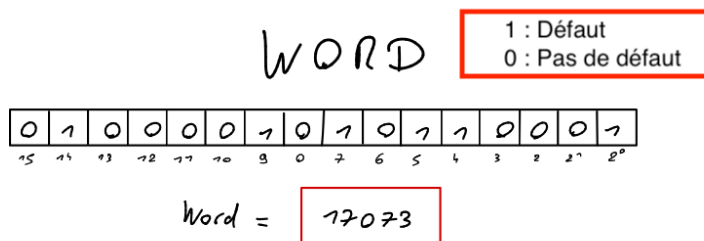
Dans cette deuxième partie, l'objectif était de récupérer la distance renvoyée par le capteur et de l'afficher sur un écran IOLink. Dans le projet deux variables étaient créées :

- Ecran_Gauche_Valeur_1_Poids_Fort
- Ecran_Gauche_Valeur_1_Poids_Faible

Si ces deux variables sont utiles, c'est parce que la valeur de la distance renvoyée par le capteur est stockée dans un « WORD », c'est-à-dire sur 16 Bits soit 2 octets. Les variables (poids_forts, poids_faibles) sont quant à elles codées sur 1 octet. Cela veut donc dire qu'à partir d'un WORD, deux BYTE sont créés et cela grâce à une union.

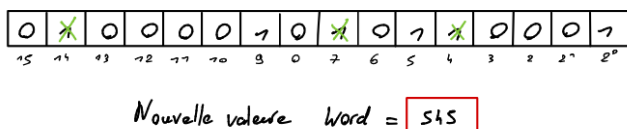
Pour plus d'explications, les unions permettent de stocker plusieurs variables de différentes tailles dans le même emplacement mémoire. Cette méthode est beaucoup utilisée dans les programmes d'arrêt d'urgence par exemple. Voici une situation dans laquelle cela peut être très utile :

Un arrêt d'urgence est déclenché par un opérateur suite à l'apparition de défauts sur une machine. Pour permettre le réarmement, le programme de gestion d'arrêt d'urgence vient lire la valeur du WORD. Dans ce WORD est stocké plusieurs BIT qui renvoient chacun 0 ou 1 en fonction de la présence d'un défaut.



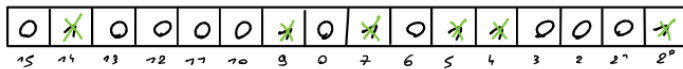
Le Bloc fonction de gestion d'arrêt d'urgence vient lire uniquement la valeur finale du WORD. Tant que cette valeur est différente de 0, le réarmement n'est pas permis car cela veut dire que certains défauts sont encore présents. L'avantage ici est claire, l'automate ne vient lire qu'une seule valeur plutôt que 16.

L'opérateur résout certains défauts :



L'opérateur peut lire les défauts présents sur l'IHM, en résoudre certains et tester un acquittement. Lorsque l'acquiescement est déclenché les valeurs de BIT de défaut changent et la valeur du WORD aussi. Cependant, tant que certains défauts sont encore présents, la valeur du WORD est toujours différente de 0. Le réarmement n'est toujours pas permis.

L'opérateur reset tous les défauts :



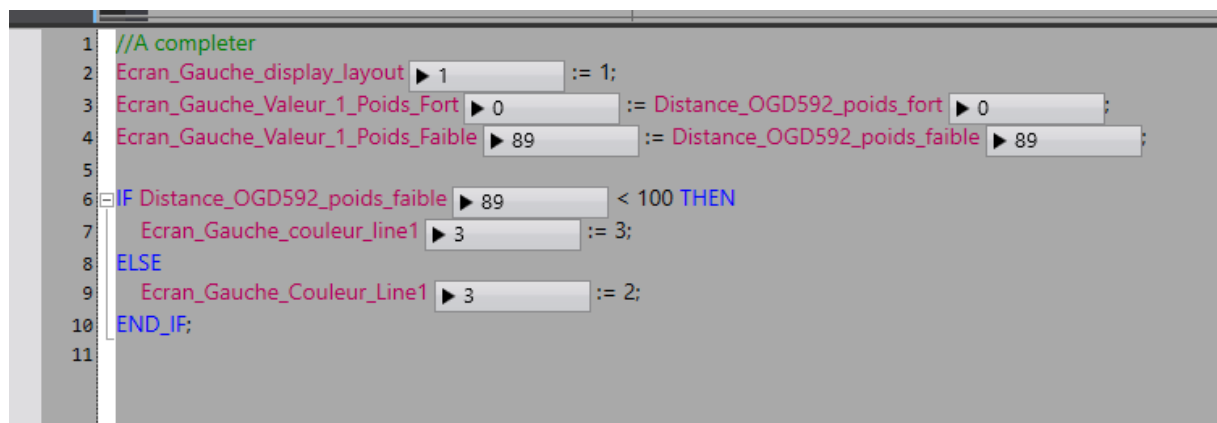
Valeur finale de WORD = 0

Maintenant que la valeur du WORD est nulle, l'automate autorise le réarmement.

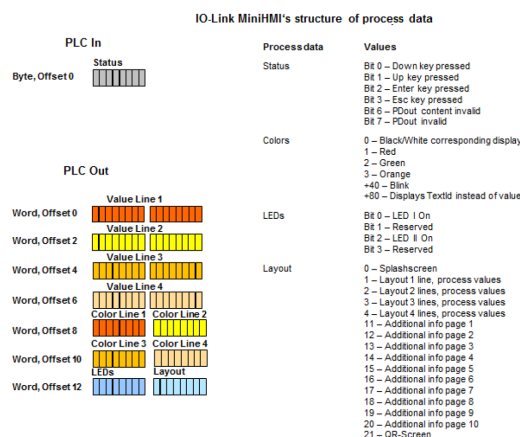
Pour revenir à notre cas, l'union est simplement utilisée car . On stock deux BYTE dans un WORD puis on vient les rechercher ensuite en définissant un BYTE de poids fort et un BYTE de poids faible. Dans notre cas, nous n'avons utilisé que le BYTE de poids faible car la distance était suffisamment faible.

Une fois qu'il était possible de visualiser la distance sur l'écran, nous avons fait en sorte de modifier la couleur de fond en fonction de la distance relevé.

Voici donc notre programme final :



On peut voir dans notre programme que l'on donne une valeur numérique à la variable de couleur. Quatre couleurs sont disponibles et sont repérables par des chiffres allant de 0 à 3.



Dans notre cas, on a donc défini que si la distance était inférieure à 100 mm, l'écran devait être orange, dans le cas contraire, l'écran est vert.

Pour la dernière partie du TP, nous devons afficher les positions (X, Y, Z) du robot. Nous y sommes parvenus en utilisant les unions.

Nous avons commencé par définir des variables en les configurant en union, les unions étaient déjà définies dans le projet, on a donc juste été les chercher.

Malheureusement nous n'avons pas de photos du code ou même du résultat sur l'écran mais le professeur avait validé le fonctionnement pendant le TP.

Conclusion :

Ce TP nous a permis de comprendre le fonctionnement des cartes IOlink, désormais nous comprenons comment les capteurs intelligents communiquent avec les automates. On a également appris des techniques qui nous permettent de communiquer vers l'automate autre chose que simplement 0 ou 1 depuis un capteur. On a également été confronté aux contraintes des trames de l'Ethercat mais ça n'a pas été une mauvaise chose car en étudiant le fonctionnement des unions, le fonctionnement des trames et du stockage mémoire s'est éclairé. La machine sur laquelle nous avons travaillé ce jour était automatisée par un automate OMRON, mon binôme et moi ne travaillons pas sur ce type d'automate dans nos entreprises respectives, c'est donc un axe d'amélioration important notamment au niveau de notre capacité à nous adapter à n'importe quel automate ou machine.